

TagProtect re-verified on reader application 5.0.7 - FXR90

TagProtect still works after the OS update. The same 5-phase test that confirmed it on 5.0.4 was re-run on 5.0.7, on the same reader and the same two tags, and reproduced **identical results in every phase**.

Reader	10.233.48.49 - FXR90, serial 25289523070378
Reader application	5.0.7 (was 5.0.4 in the original test)
Antennas	5 and 6 (1-4 disconnected on this unit)
Power	30 dBm
Date	2026-09-08
Tags	2 x Impinj Monza 4i - same physical tags, TIDs byte-identical
Password	12348765 - already on both tags, nothing written
Result	[OK] No regression. All five phases match 5.0.4 exactly.

Original test: `../tagprotect-CONFIRMED-monza4i/TAGPROTECT_CONFIRMED.md`

1. Headline - 5.0.7 vs 5.0.4 side by side

Phase	Gen2X applied	Target 5.0.4	Target 5.0.7	Control (both)
1. Baseline	none	5/5	5/5	5/5
2. enableTagProtection	tagProtect	1/5	1/5	5/5
3. Gen2X cleared entirely	none	0/5	0/5	5/5
4. enableTagVisibility	tagProtect	5/5	5/5	5/5
5. disableTagProtection	none	5/5	5/5	5/5

Five 10-second inventories per phase, 25 inventories total. Every number is the same on both firmware versions - including the single run-1 hit in phase 2.

The control tag `e28011b0a505007698ca0895` - an identical Monza 4i never named in any request - stayed visible **5/5 in all five phases**, isolating the effect to the targeted EPC.

Phase 3 is the decisive one. With **no Gen2X configuration loaded at all** and a plain `PUT /cloud/start {}`, the target is still invisible 0/5. The protection lives in the tag's silicon, not in reader-side filtering.

Raw per-run counts: `evidence/results.csv`, and one `.ndjson` capture per inventory under `evidence/p1...p5`.

2. Same tags, verified by TID

Both tags were re-identified before testing rather than assumed:

EPC	TID	MDID	TMN	Chip
e28011b0a5050076c4d7530a (target)	e28011b02000b30a26ba03b6	0x801	0x1B0	Monza 4i
e28011b0a505007698ca0895 (control)	e28011b02000a895c65003b4	0x801	0x1B0	Monza 4i

Both TIDs are **byte-identical to the 5.0.4 run**, so these are physically the same two tags - the comparison is a true re-verification, not a re-test on different hardware. Capture: `evidence/tid_scan.ndjson`.

TID decode: byte 0 `0xE2` = Class-1 Gen-2; bytes 1-2 upper 12 bits = MDID (`0x801` = Impinj); bytes 2-3 lower 12 bits = TMN (`0x1B0` = Monza 4i).

3. The password was already there, and is unchanged

Read from RESERVED words 0-3 before and after the whole run:

```
GET RESERVED w0-3 -> "1234876512348765"
kill password (w0-1) = 12348765
ACCESS password (w2-3) = 12348765
```

	Before	After
Target access password	12348765	12348765 [OK]
Target kill password	12348765	12348765 [OK]
Control access password	12348765	12348765 [OK]

Nothing was written to either tag's memory at any point. The existing password was read and used. This keeps the write step out of the experiment as a variable. Captures: `evidence/pw_scan.ndjson` (before), `evidence/pw_after.ndjson` (after).

4. The sequence run

4.1 Route tag data to a readable endpoint

The reader was publishing to a disconnected `WEBSOCKET_TEST` endpoint. Repointed to the local MQTT broker (`10.117.229.18:1883`, topic `fxr90-49/tevents`).

5.0.7 accepted the same payload 5.0.4 required. The original test noted 5.0.4 rejected two simpler shapes with `invalid "additional" JSON object` and needed the `additionalOptions` wrapper. The wrapper form was reused here and accepted first time - so this is not a confirmation that 5.0.7 has relaxed, only that the stricter shape still works. Untested on 5.0.7: whether the simpler shapes now pass.

4.2 Baseline - 5 plain scans

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[5,6],"transmitPower":[30,30],"tagMetaData":["ANTENNA","RSSI"]}
```

```
PUT /cloud/start {} x5, 10 s each
```

Target **5/5**, control **5/5**.

4.3 Protect the target

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagProtection",
  "password":"12348765",
  "tagID":"e28011b0a5050076c4d7530a",
  "enableShortRange":false}}
```

-> 200, "Success: Gen2X configured. Use `applyImpinjGen2X` flag in start command to apply features." - read back verbatim.

```
PUT /cloud/start {"applyImpinjGen2X":true} x5, 10 s each
```

Target **1/5** (run 1 only), control **5/5**.

The run-1 hit is the protect operation reaching the tag mid-inventory - the tag answers once, then goes silent. Runs 2-5 are all misses. Same behaviour as 5.0.4.

4.4 The decisive check - clear Gen2X and rescan

```
PUT /cloud/impinjGen2X {"fastID":{"enabled":false}}
PUT /cloud/start {} x5
```

Target **0/5**, control **5/5**. No Gen2X config, no apply flag, and the tag is still gone.

4.5 Read the protected tag with the password

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagVisibility","password":"12348765"}}

PUT /cloud/start {"applyImpinjGen2X":true} x5
```

Target **5/5**, control 5/5 - a protected tag is invisible to everyone except a reader supplying the correct password.

4.6 Unprotect

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"disableTagProtection",
               "password":"12348765",
               "tagID":"e28011b0a5050076c4d7530a"}}

PUT /cloud/start {"applyImpinjGen2X":true}
```

Then Gen2X cleared and five plain scans: target **5/5**, control 5/5. Fully reversible.

5. What this confirms on 5.0.7

Claim	5.0.7
enableTagProtection makes a Gen2X tag RF-silent	[OK] confirmed - 5/5 -> 0/5
Protection is written to the tag, not the reader	[OK] confirmed - persists with no Gen2X loaded
enableTagVisibility lets an authorised reader read it	[OK] confirmed - 0/5 -> 5/5
disableTagProtection reverses it	[OK] confirmed - back to 5/5
The effect is specific to the targeted EPC	[OK] confirmed - control 5/5 in all 5 phases
Two-step configure-then-apply is required	[OK] confirmed
No regression from 5.0.4 -> 5.0.7	[OK] confirmed - identical in all phases

6. [BUG] Separate finding - PUT /cloud/mode hung until reboot

Not a TagProtect issue, but it blocked this test and is worth reporting.

On first contact after the 5.0.4 -> 5.0.7 OS update, **every** `PUT /cloud/mode` hung indefinitely with no HTTP response, while every other endpoint answered normally:

Request	Result
GET /cloud/mode	[OK] 200 in 0.048 s
GET /cloud/version	[OK] 200

GET /cloud/status	[OK] 200 in ~4 s
PUT /cloud/stop	[OK] 422 in 0.048 s (RFID inventory is not running)
PUT /cloud/impinjGen2X	[OK] 200 in 0.055 s
PUT /cloud/start	[OK] 200 in 0.151 s
PUT /cloud/mode	[X] no response - 45 s, 60 s, 90 s and 120 s timeouts all expired

Ruled out:

- **Payload shape** - {"type": "SIMPLE"}, the simplest legal body, hung the same as the full CUSTOM-with-accesses payload.
- **Token expiry** - a 401 was seen once and fixed by minting per request; the hangs continued with fresh tokens and produce no response at all, not a 401.
- **Radio busy** - the radio was confirmed **inactive** and PUT /cloud/stop returned "not running" before the attempt.
- **Reader health** - CPU 8%, temperature 35 °C, antennas 5 and 6 connected.

Corroborating symptom: GET /cloud/status reported "operatingMode": None throughout, while GET /cloud/mode returned a stored CUSTOM config. With the mode unset, PUT /cloud/start returned 200 and radioActivity went active, but **no tag events were published at all** - the radio ran with no operating mode.

Recovery: PUT /cloud/reboot -> 200. Reader back in **130 s**. The very next PUT /cloud/mode returned **200 in 0.244 s**, and all 25 subsequent inventories worked normally.

So this is a **recoverable stuck state, not a permanent defect** - but it is a silent one. A client gets no error, only a hang, and a started radio that reports nothing.

Evidence: cloud-mode-PUT/19-PUT-mode-HANGS-until-reboot-BUG/ - response body/evidence_before_reboot.txt - one capture showing every endpoint's status and timing side by side.

Questions for Zebra:

1. Is PUT /cloud/mode expected to be able to block with no response and no timeout? A 5xx or a 503 would let a client react.
2. Was this a residue of the OS update? The reader had been up since the 5.0.4 -> 5.0.7 flash and had not had a mode write since.
3. Should PUT /cloud/start succeed when operatingMode is unset? Returning 200 and activating the radio while publishing nothing is hard to diagnose.

Caveat: observed once, on one reader, immediately after an OS update. Not reproduced deliberately - the reboot cleared it and it did not recur across the rest of the session.

7. Reader and tag state after testing

	Before	After	
Target visibility	5/5	5/5	[OK] restored
Control visibility	5/5	5/5	[OK] untouched
Target access password	12348765	12348765	[OK] unchanged
Target kill password	12348765	12348765	[OK] unchanged
impinjGen2X config	{"fastID":{"enabled":false}}	{"fastID":{"enabled":false}}	[OK] idle
Reader mode	CUSTOM, ant [5,6], 30 dBm	CUSTOM, ant [5,6], 30 dBm	[OK] restored
Data endpoint	WEBSOCKET_TEST	WEBSOCKET_TEST	[OK] restored
Radio	inactive	inactive	[OK]

Both tags left unprotected and fully readable. One deliberate change to the reader: it was **rebooted** to clear the mode-write hang in §6.

8. Files

```
tagprotect-REVERIFIED-507/
|-- TAGPROTECT_REVERIFIED_507.md      <- this file
`-- evidence/
    |-- results.csv                   per-phase target/control counts
    |-- tid_scan.ndjson               chip identification
    |-- pw_scan.ndjson                access password before
    |-- pw_after.ndjson               access password after
    |-- mode_tid.json                 TID read mode payload
    |-- mode_pw.json                  RESERVED read mode payload
    |-- mode_plain.json               plain inventory mode payload
    |-- g2x_protect.json              enableTagProtection payload
    |-- scan.sh                       one inventory + MQTT capture
    |-- phase.sh                      five inventories, counts hits
    |-- p1_baseline/                  run1-5.ndjson + .log
    |-- p2_protected/                 run1-5.ndjson + .log
    |-- p3_cleared/                   run1-5.ndjson + .log    <- the decisive phase
    |-- p4_visibility/                run1-5.ndjson + .log
    `-- p5_unprotected/               run1-5.ndjson + .log
```

Recount any phase from the raw captures:

```
grep -c e28011b0a5050076c4d7530a evidence/p3_cleared/run*.ndjson # target: all 0
grep -c e28011b0a505007698ca0895 evidence/p3_cleared/run*.ndjson # control: all 1
```